

Практическое занятие № 3

Цель: изучить элементы интерфейса настольного приложения (таблица, список, счетчик, слайдер)

Время выполнения: 1 пара

Формат сдачи: показать преподавателю вашей подгруппы приложение с элементами и вашими пояснениями методов + конспект

Задание:

1. Изучить теоретический материал для каждого элемента и примеры.
2. Переписать примеры, запустить, рассмотреть принцип работы элемента и прокомментировать каждую строку. Выполнить задания там, где они указаны.
3. Написать небольшой конспект (все только по сути, без воды)
4. Показать сделанные результаты занятия преподавателю вашей подгруппы, ответить на вопросы.

Теоретический материал:

Таблица:

QTableWidget - позволяет сделать таблицу (со строками и столбцами). Для простого отображения данных в табличном формате не потребуется много настроек. Также для просмотра данных в табличном формате можно использовать QTableView - представляет больше настроек по отображению данных. В данной части рассмотрим пример с QTableWidget.

```

1  from PySide6.QtGui import QColor
2  from PySide6.QtWidgets import (QApplication, QTableWidgetItem,
3  |                               QTableWidgetItem)
4
5  colors = [("Red", "#FF0000"),
6  |          ("Green", "#00FF00"),
7  |          ("Blue", "#0000FF"),
8  |          ("Black", "#000000"),
9  |          ("White", "#FFFFFF"),
10 |          ("Electric Green", "#41CD52"),
11 |          ("Dark Blue", "#222840"),
12 |          ("Yellow", "#F9E56d")]
13
14  def get_rgb_from_hex(code):
15      code_hex = code.replace("#", "")
16      rgb = tuple(int(code_hex[i:i+2], 16) for i in (0, 2, 4))
17      return QColor.fromRgb(rgb[0], rgb[1], rgb[2])
18
19  app = QApplication([])
20  table = QTableWidgetItem()
21  table.setRowCount(len(colors))
22  table.setColumnCount(len(colors[0]) + 1)
23  table.setHorizontalHeaderLabels(["Name", "Hex Code", "Color"])
24
25  for i, (name, code) in enumerate(colors):
26      item_name = QTableWidgetItem(name)
27      item_code = QTableWidgetItem(code)
28      item_color = QTableWidgetItem()
29      item_color.setBackground(get_rgb_from_hex(code))
30      table.setItem(i, 0, item_name)
31      table.setItem(i, 1, item_code)
32      table.setItem(i, 2, item_color)
33
34  table.show()
35  app.exec()

```

Задание: создайте таблицу “Пользователи”, в которой разместить следующую информацию: Фамилия, Имя, Отчество, Дата рождения, Адрес электронной почты. Заполните в коде эту таблицу данными и отобразите в окне приложения. В процессе написания кода используйте классы, конструкторы, поля. Сохраните получившийся код, в конце пары - покажите его преподавателю вашей подгруппы. При необходимости дайте пояснения.

Список:

QListWidget - очень похож на QComboBox, отличается в основном доступными сигналами и своим видом на экране. Это поле со списком опций, а не раскрывающийся список.

```

1  from PySide6.QtWidgets import (
2      QApplication,
3      QListWidget,
4      QMainWindow,
5  )
6
7  class MainWindow(QMainWindow):
8      def __init__(self):
9          super(MainWindow, self).__init__()
10
11         self.setWindowTitle("Список")
12
13         listwidget = QListWidget()
14         listwidget.addItem("Первый элемент")
15         listwidget.addItem("Второй элемент")
16         listwidget.addItem("Третий элемент")
17         listwidget.currentItemChanged.connect(self.index_changed)
18         listwidget.currentTextChanged.connect(self.text_changed)
19
20         self.setCentralWidget(listwidget)
21
22     def index_changed(self, index):
23         print(index) # попробуйте посмотреть вариант: index.text() - это будет тот же самый объект QListWidgetItem
24
25     def text_changed(self, text):
26         print(text)
27
28 app = QApplication([])
29 window = MainWindow()
30 window.show()
31 app.exec()

```

QListWidget предлагает **currentItemChanged** сигнал, который отправляет QListWidgetItem(элемент списка), и **currentTextChanged** сигнал, который отправляет текст элемента.

Счетчик:

QSpinBox - предоставляет небольшое поле числового ввода со стрелками для увеличения и уменьшения значения. QSpinBox поддерживает целые числа, тогда как виджет, QDoubleSpinBox, поддерживает числа с плавающей точкой.

```

1  from PySide6.QtWidgets import (
2      QApplication,
3      QSpinBox,
4      QMainWindow,
5  )
6
7  class MainWindow(QMainWindow):
8      def __init__(self):
9          super().__init__()
10
11         self.setWindowTitle("Счетчик")
12
13         spinbox = QSpinBox()
14         # Или: doublespinbox = QDoubleSpinBox()
15
16         spinbox.setMinimum(-10)
17         spinbox.setMaximum(100)
18         # Или: doublespinbox.setRange(-10, 100)
19
20         # Русский текст поменять на "Сумма" или придумать самостоятельно
21         # подходящий по смыслу (увеличение или уменьшение чего-либо дробного)
22         spinbox.setPrefix("Количество игроков: ")
23         spinbox.setSuffix(" чел.")
24         spinbox.setSingleStep(1) # или 0.5 для QDoubleSpinBox
25         spinbox.valueChanged.connect(self.value_changed)
26         spinbox.textChanged.connect(self.value_changed_str)
27
28         self.setCentralWidget(spinbox)
29
30         def value_changed(self, value):
31             print(value)
32
33         def value_changed_str(self, str_value):
34             print(str_value)
35
36     app = QApplication([])
37     window = MainWindow()
38     window.show()
39     app.exec()

```

Оба QSpinBox и QDoubleSpinBox имеют .valueChanged сигнал, который срабатывает всякий раз, когда изменяется их значение. Необработанный .valueChanged сигнал отправляет числовое значение (либо int, либо float), а .textChanged отправляет значение в виде строки, включая символы префикса и суффикса. При желании можно отключить ввод текста в строке редактирования счетчика, установив его в режим «только для чтения».

```

1  from PySide6.QtWidgets import (
2      QApplication,
3      QSpinBox,
4      QMainWindow,
5  )
6
7  class MainWindow(QMainWindow):
8      def __init__(self):
9          super().__init__()
10
11         self.setWindowTitle("Счетчик")
12
13         spinbox = QSpinBox()
14         # Или: doublespinbox = QDoubleSpinBox()
15
16         spinbox.setMinimum(-10)
17         spinbox.setMaximum(100)
18         # Или: doublespinbox.setRange(-10, 100)
19
20         # Русский текст поменять на "Сумма" или придумать самостоятельно
21         # подходящий по смыслу (увеличение или уменьшение чего-либо дробного)
22         spinbox.setPrefix("Количество игроков: ")
23         spinbox.setSuffix(" чел.")
24         spinbox.setSingleStep(1) # или 0.5 для QDoubleSpinBox
25         spinbox.lineEdit().setReadOnly(True)
26         spinbox.valueChanged.connect(self.value_changed)
27         spinbox.textChanged.connect(self.value_changed_str)
28
29         self.setCentralWidget(spinbox)
30
31         def value_changed(self, value):
32             print(value)
33
34         def value_changed_str(self, str_value):
35             print(str_value)
36
37     app = QApplication([])
38     window = MainWindow()
39     window.show()
40     app.exec()

```

Слайдер:

QSlider - предоставляет виджет-слайдер, который внутренне работает так же, как QDoubleSpinBox. Вместо того, чтобы отображать текущее значение в числовом виде, ползунок определяет свое значение по положению своей ручки по длине виджета. Это часто полезно при предоставлении регулировки между двумя крайностями, но когда абсолютная точность не требуется.

Существует дополнительный .sliderMoved сигнал, который срабатывает всякий раз, когда ползунок перемещается, и .sliderPressed сигнал, который издается всякий раз, когда на ползунок нажимают.

```

1  from PySide6.QtWidgets import (
2      QApplication,
3      QSlider,
4      QMainWindow,
5  )
6
7  class MainWindow(QMainWindow):
8      def __init__(self):
9          super().__init__()
10
11         self.setWindowTitle("Слайдер")
12
13         slider = QSlider()
14
15         slider.setMinimum(-10)
16         slider.setMaximum(10)
17         # Или: widget.setRange(-10,3)
18
19         slider.setSingleStep(1)
20
21         slider.valueChanged.connect(self.value_changed)
22         slider.sliderMoved.connect(self.slider_position)
23         slider.sliderPressed.connect(self.slider_pressed)
24         slider.sliderReleased.connect(self.slider_released)
25
26         self.setCentralWidget(slider)
27
28         def value_changed(self, value):
29             print(value)
30
31         def slider_position(self, position):
32             print("позиция (значение): ", position)
33
34         def slider_pressed(self):
35             print("Нажат!")
36
37         def slider_released(self):
38             print("Отпущен!")
39
40     app = QApplication([])
41     window = MainWindow()
42     window.show()
43     app.exec()

```

Для того, чтобы повернуть вертикально или горизонтально виджет необходимо указать:

```
1  from PySide6.QtWidgets import (
2      QApplication,
3      QSlider,
4      QMainWindow,
5  )
6  from PySide6.QtCore import Qt
7
8  class MainWindow(QMainWindow):
9      def __init__(self):
10         super().__init__()
11
12         self.setWindowTitle("Слайдер")
13
14         slider = QSlider(Qt.Orientation.Horizontal)
15
16         slider.setMinimum(-10)
17         slider.setMaximum(10)
18         # Или: widget.setRange(-10,1)
19
20         slider.setSingleStep(1)
21
22         slider.valueChanged.connect(self.value_changed)
23         slider.sliderMoved.connect(self.slider_position)
24         slider.sliderPressed.connect(self.slider_pressed)
25         slider.sliderReleased.connect(self.slider_released)
26
27         self.setCentralWidget(slider)
28
29         def value_changed(self, value):
30             print(value)
31
32         def slider_position(self, position):
33             print("позиция (значение): ", position)
34
35         def slider_pressed(self):
36             print("Нажат!")
37
38         def slider_released(self):
39             print("Отпущен!")
40
41     app = QApplication([])
42     window = MainWindow()
43     window.show()
44     app.exec()
```

Задание: Создайте слайдер, который будет лежать в диапазоне от 0 до 100. В Текстовый блок (лейбл) выводите текущее значение слайдера.